SOFTWARE REQUIREMENTS SPECIFICATION

RealmForge

Team 20
Aeolus Software

Andy Huynh
Owen Johnson
Christopher Ludwig
Andreas Sideris
Humna Saeed

Revision 0: October 16, 2026

Revision History

Name	Date	Reason For Changes	Version
Andy Huynh	2026-10-05	First Draft	1.0
Andy Huynh	2026-10-13	Second draft of G4	1.0
Andreas Sideris	2026-10-10	P3 and P4 drafts	1.0
Andreas Sideris	2026-10-12	E3 draft	1.0
Andreas Sideris	2026-10-13	E2 draft	1.0
Andreas Sideris	2026-10-15	AI disclosure and Reflection updated	1.0
Owen Johnson	2026-10-15	Reflection updated	1.0

Document Template and Conventions

Document Template:

This document follows the requirements documentation structure presented in the Handbook of requirements and business analysis, by Bertrand Meyer. The document template has been modified with the following changes to better suit the project's needs:

- A requirement and behavior artifact traceability table has been added to section P.7, to enable further traceability between essential requirements and behavior.

Requirement and Behavior Artifact Conventions:

All formal requirements in this document will use the following conventions:

- Functional Requirements are given the unique identifier "SFR-XXX", meaning "Software Functional Requirement", with "XXX" being a unique numerical identifier.
- Non-Functional Requirements are given the unique identifier "SNFR-XXX", meaning "Software Non-Functional Requirement", with "XXX" being a unique numerical identifier.
- Environmental Requirements are given the unique identifier "ENV-XXX", meaning "Environmental Requirement", with "XXX" being a unique numerical identifier.
- Mandatory Requirements that must be satisfied at the conclusion of the project (except those that are denoted as "stretch" requirements to be satisfied in a future revision of the software) use the keyword "**shall**" to express a requirement. For example: "The system shall ___". The majority of requirements presented in this document will follow this convention.
- Non-Mandatory Requirements that express a recommendation for the implementation of the system use the keyword "**should**" to express these recommendations. For example: "The system should ___".

Behavior Artifacts in this document will use the following conventions:

- High-level Use Cases are given the unique identifier "UC-XXX", meaning "Use Case", with "XXX" being a unique numerical identifier.

- Detailed User Scenarios are given the unique identifier "US-XXX", meaning "User Scenario", with "XXX" being a unique numerical identifier.

The following traceability relationships will be used between requirements and behavior artifacts:

- A requirement can "**satisfy**" another requirement or be "**satisfied by**" another requirement. This relationship describes when a requirement provides further detail and more criteria to **satisfy** another, such as a functional requirement providing more detail to satisfy a non-functional requirement.
- A requirement can "**refine**" a use case or user scenario, and a use case or user scenario can be "**refined by**" a requirement. This relationship describes when a requirement expresses a mandatory need to implement the behavior described in a use case or user scenario in a specific way, by **refining** the behavior.

Contents

Goals book **6**
G.1 Context and overall objective . . . 6
G.2 Current situation . . . 6
G.3 Expected benefits . . . 6
G.4 Functionality overview . . . 7
G.5 High-level usage scenarios . . . 7
G.6 Limitations and exclusions . . . 9
G.7 Stakeholders and requirements sources . . . 9

Environment book **10**
E.1 Glossary . . . 10
E.2 Components . . . 13
E.3 Constraints . . . 14
E.4 Assumptions . . . 15
E.5 Effects . . . 16
E.6 Invariants . . . 16

System book **17**
S.1 Components . . . 17
S.2 Functionality . . . 17
S.3 Interfaces . . . 17
S.4 Detailed usage scenarios . . . 17
S.5 Prioritization . . . 18
S.6 Verification and acceptance criteria . . . 18

Project book **19**
P.1 Roles and personnel . . . 19
P.2 Imposed technical choices . . . 19
P.3 Schedule and milestones . . . 19
P.4 Tasks and deliverables . . . 20
P.5 Required technology elements . . . 22
P.6 Risks and mitigation analysis . . . 22
P.7 Requirements process and report . . . 22

Appendix **23**
Appendix — Generative AI Use Disclosure . . . 23
Appendix — Reflection . . . 24

Goals

Contents

G.1 Context and overall objective 6
G.2 Current situation 6
G.3 Expected benefits 6
G.4 Functionality overview 7
G.5 High-level usage scenarios 7
G.6 Limitations and exclusions 9
G.7 Stakeholders and requirements sources 9

Comment: *Goals are "needs of the target organization, which the system will address". While the development team is the principal user of the other books, the Goals book addresses a wider audience: essentially, all stakeholders.*

G.1 Context and overall objective

Comment: *High-level view of the project: organizational context and reason for building a system.*
Comment: *This chapter should not be empty!*

Goal 1. This is a goal example. If you need explicit (and automatic) numbering, you can use the definitions in the `.tex` template. Is is refined by 3

Requirement 2. *This is a requirement example. It illustrates how numbering is continuous and cross-types (if this is what you need).*

G.2 Current situation

Comment: *Current state of processes to be addressed by the project and the resulting system.*

Requirement 3. *This is a requirement example. It refines 1*

G.3 Expected benefits

Comment: *New processes, or improvement to existing processes, made possible by the projectâ€™s results.* **Comment:** *This chapter should not be empty!*

G.4 Functionality overview

Functional Overview

Setting properties: The system application starts up and the user will be shown options that will allow the user to pick and change various settings during the start of the game. This will allow the user to experiment with different variants of the game which in turn alters the experience of the user with game.

Traversal: The system will display the HUD as well other UI components such as the main game screen and travel menu. This will allow the player to navigate to different areas that contain various rewards and opponents. The player will be shown the possible risks and rewards before deciding to enter a certain area.

Combat: The player will enter combat with an enemy when entering a new zone or area. The system will transition into a combat phase to which the player faces the adversary. Once in combat, a battle screen will appear as well as the player's in-game character and enemy sprite. The player will be shown various options to deal with the encounter. For example, if the player chooses to attack the enemy, an attack action is displayed followed by the enemies opposing attack.

Reward: If the player defeats the enemy, combat ends and the player will be rewarded with resources used for currency later on.

End of Run: If the player character's health points reaches zero during combat, the player will die and be returned to the main hub/starting area. The player will be able to spend resources and start the main gameplay loop again via traversal.

Spending Resources: When the player is in the main hub, the player shall be allowed to spend the resources accumulated throughout the previous journies to construct buildings and upgrades that will aid the player for the next gameplay run.

Non-Functional Overview

Ease of use: The system will display hints and instructions that will allow the player to navigate and play the game with little or no external help.

Dynamic difficulty and experience: Changing the initial properties of the game should let the player handle how difficult the game is based on formal experience. Continuously altering properties should keep the gameplay from being stale and repeatitive.

G.5 High-level usage scenarios

Use Case 001 (UC-001): Selecting Properties

1. User launches the RealmForge software.
2. User selects feature variation configurations.
3. User decides which features of the game to be changed.
4. User finishes setting up by beginning the game.

Use Case 002 (UC-002): Playing a full session

1. User selects to begin a new game session.
2. User enters the main game world.
3. User decides which area of the game world to go to explore based on the information given for each area.
4. User selects which area to go to and begins traveling.
5. User enters a new area and begins an encounter with an enemie/enemies (see UC-003).
6. User is successful (continue below) or fails in the encounter (see UC-005).
7. User continues traveling the game world (repeats step 2 through 6).
8. User completes the win condition for the game session (see UC-005).

Use Case 003 (UC-003): Battling enemies

1. User enters an encounter with the enemie/enemies.
2. User selects attacks to defeat the enemie/enemies.
3. User repeats step 2 until the enemie/enemies are defeated.
4. User collects rewards for successfully completing the encounter.
5. User returns to game's world map.

Use Case 004 (UC-004): Upgrading and purchasing items

1. User enters town/city/central hub.
2. User chooses and selects an available option(s) for purchasing.
3. User confirms the purchase.
4. User repeats steps 2 and 3 until the user is satisfied or resources are spent.
5. User leaves the town/city/central hub to continue the game session.

Use Case 005 (UC-005): Continuing or start a new game

1. User completes a game session or fails in between (see UC-002).
2. User is given an option to return to the central hub (see UC-004) or quit the current game session (see below).
3. User selects to change features (see UC-001) or proceeds with the same configurations.
4. User begins a new game.

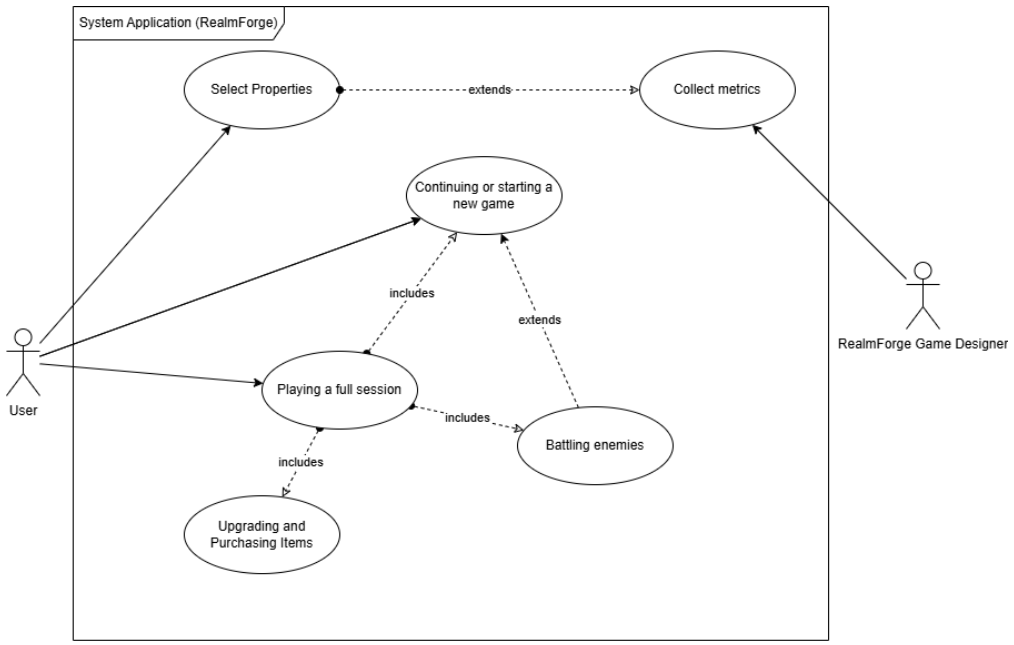


Figure 1: Use Case Diagram

G.6 Limitations and exclusions

Comment: *Aspects that the system need not address.*

G.7 Stakeholders and requirements sources

Comment: *Groups of people who can affect the project or be affected by it, and other places to consider for information about the project and system.* **Comment:** *This chapter should not be empty!*

Environment

Contents

E.1 Glossary 10
E.2 Components 13
E.3 Constraints 14
E.4 Assumptions 15
E.5 Effects 16
E.6 Invariants 16

Comment: *The Environment book describes the application domain and external context, physical or virtual (or a mix), in which the system will operate.*

E.1 Glossary

Comment: *Clear and precise definitions of all the vocabulary specific to the application domain, including technical terms, words from ordinary language used in a special meaning, and acronyms. This chapter should not be empty!*

- **User** - The main stakeholder that is operating the system; providing inputs and receiving outputs.
- **Player** - Is another name refering the user.
- **User-Interface (UI)** - The entire set of displayed service elements the player used to navigate the system.
- **Heads-Up-Display (HUD)** - The set of interface elements displayed ontop of the actually gameplay that contains vital information helpful to the player or user.
- **Central Processing Unit (CPU)** - The central processor for a computer.
- **Streaming Single Instruction, Multiple Data Extentions 4 (SSE4.2)** - A processor instruction set used by most modern processors.
- **Advanced Micro Devices (AMD)** - An American semiconductor company that produces processors.
- **Graphics Processing Unit (GPU)** - A computer's processor for displayed graphics.

- **Open Graphics Library (OpenGL)** - An application programming interface used to render computer graphics.
- **NVIDIA Corporation** - An American technology company that produces graphics processors.
- **Graphics Core Next (GCN)** - A series of graphics processor architectures that is used in AMD graphics processors.
- **Random Access Memory (RAM)** - Short-term memory used by a computer's processor.
- **Frames per Second (FPS)** - The number of graphical frames displayed by a computer program per second.
- **Personal Computer (PC)** - A computer owned by individuals, for personal use.
- **Verification and Validation (VnV)** - The process of ensuring that a design satisfies its requirements, and its requirements are aligned with the design's purpose (generally).
- **Feature-Oriented Domain Analysis (FODA)** - A domain analysis method, focusing on features of a design and their variations.
- **Proof of Concept (POC)** - An initial version of a design, intended to demonstrate the possibility of its implementation without being a full prototype.

  ¡¡¡¡¡¡¡ HEAD
- **Sprite** - An in-game 2-dimensional picture representing an object in the 2D-world.
  =======
- **GDUnit4** - A Godot module that can be added to any base godot project. The module allows for the creation of unit test cases.
- **Inventory** - The set of resources, items, and currency store within code and available to access for the player ¿¿¿¿¿¿¿ main
- **GDLint** - A Godot module that can be added to any base godot project. The module is a linting tool to help will proper coding styles and errors for the Godot language.
- **Integrated Development Environment** - A set of software applications and tools within a programmer's arsenal to help develop software code efficiently.
- **Operation System** - A collection of software that manages a computer's hardware and software applications by handling allocation of resources.
- **LaTeX** - LaTeX is a high-quality typesetting system; it includes features designed for the production of technical and scientific documentation.

- **Continuous Integration / Continuous Development** - An iterative approach to software development where updates are pushed in small increments and re-evaluated for further development

E.2 Components

Included below is a System Context Diagram, outlining at a high level how the system will interact with external components:

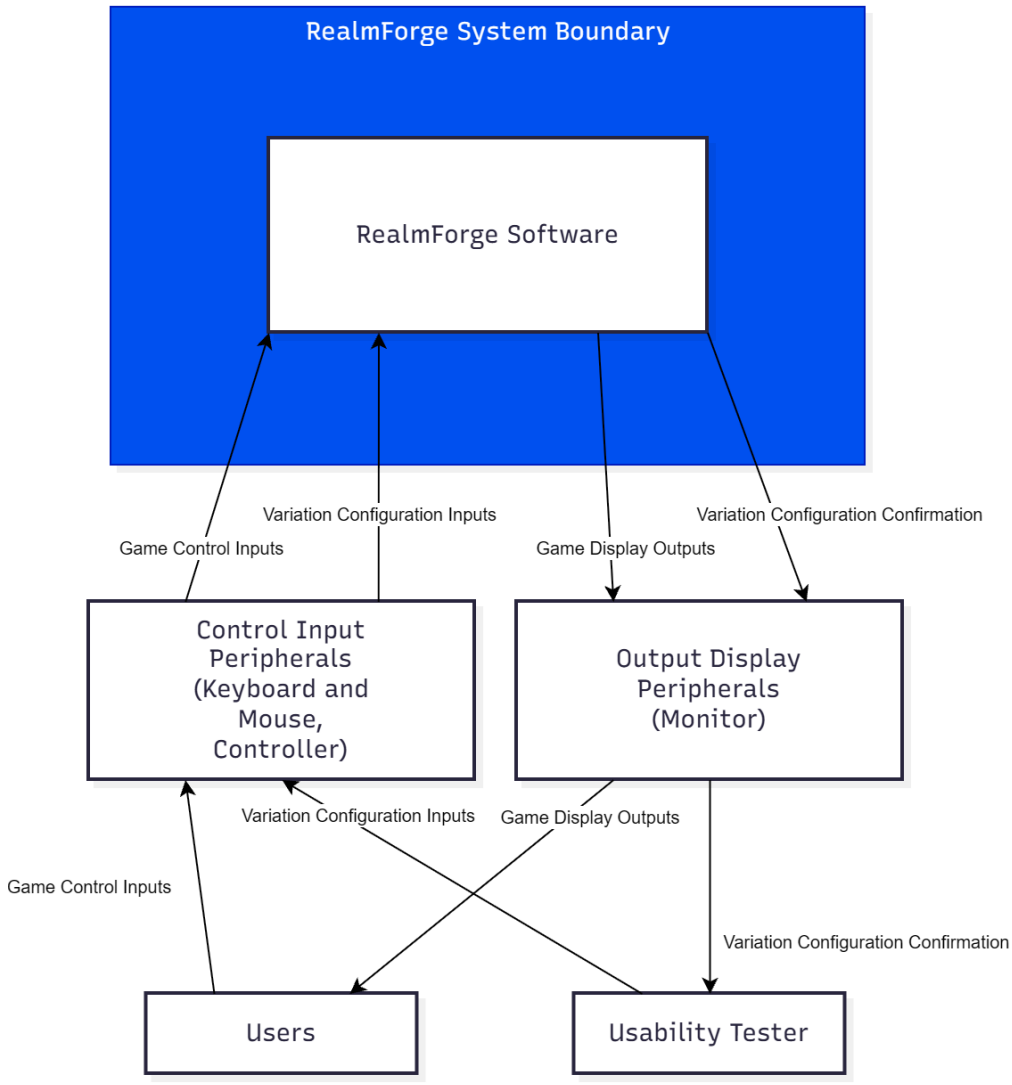


Figure 2: RealmForge System Context Diagram

The RealmForge software (also denoted as the system) will require the use of the following external components to operate successfully:

- **Control Input Peripherals:** Devices used to provide input to the system. This could include a keyboard and mouse, or a controller used to provide input. The system will require inputs from these devices to affect the state of the video game it

executes (during gameplay), and to configure the system's feature variation points during testing.

- **Output Display Peripherals:** A device used to display outputs provided by the system. This could be fulfilled by a monitor, or another display device (such as a television). The system will require this device to display graphical outputs during gameplay (video game graphics), as well as confirmation of feature variation configurations (variation config. menu graphics).

E.3 Constraints

ENV-001: The system shall be compatible with user machines running Windows 10 or 11 (64-bit) operating systems that meet the following recommended specifications:

- **CPU:** x86_64 CPU with SSE4.2 support, with 4 physical cores or more. Example: Intel Core i5-6600K, AMD Ryzen 5 1600.
- **GPU:** Dedicated graphics GPU with full OpenGL 4.6 support. Example: NVIDIA GeForce GTX 650, AMD Radeon HD 7750 (GCN 1.0).
- **RAM:** A minimum of 8 GB of RAM.

Rationale: This environment constraint is imposed to focus development of RealmForge on the most commonly used operating system for PC video games, Windows. The system may be extended to support other operating systems in the future if possible. The recommended specifications are included to ensure user machines can fully execute the system, accompanying any future decisions to extend its capabilities.

Fit Criterion: The system can be compiled and executed on machines running Windows 10 or 11 (64-bit) operating systems, meeting the recommended specifications outlined above, at a stable 60 fps.

Traceability: N/A

ENV-002: The system shall provide user input capabilities for keyboard and mouse peripherals, as well as common controllers (PlayStation, Xbox).

Rationale: The system must accommodate the use of various input methods (keyboard and mouse, different controllers) to facilitate usability studies incorporating these varying input methods.

Fit Criterion: The system can receive all game inputs required for its successful operation from the following input methods: USB Keyboard and Mouse, Xbox Series X/S, Xbox One, or Xbox 360 controllers, and PlayStation DualSense or DualShock 4 controllers.

Traceability: N/A

ENV-003: The system shall provide at least one explicit feature variation capability, that can be selected without recompiling the system's executable file.

Rationale: This constraint is imposed by the purpose of the development project (outlined in the Problem Statement document). The system must provide explicit feature variation capabilities to facilitate usability testing that will focus on the effectiveness of different variations.

Fit Criterion: The system provides at least one explicit feature variation capability, that can be selected and managed at run time, while the system is executing.

Traceability: N/A

E.4 Assumptions

Display Device

It is assumed that players have access to a properly connected and functioning display device that is capable of displaying RealmForge's graphics.

Input Devices

It is assumed that players have access to properly connected and functioning input devices compatible with their system, such as a keyboard and mouse or a supported controller, to interact with the game.

Operating System and Driver Support

It is assumed that the user's operating system has the necessary drivers installed and configured to support the input and display devices used by RealmForge.

Application Permissions

It is assumed that users have permission to install and run RealmForge on their devices, as well as access any local files needed to save game progress or change game settings.

Available System Resources

It is assumed that the user's device has enough available memory and processing power to run RealmForge without significant interference from other applications running at the same time.

E.5 Effects

Comment: *Elements and properties of the environment that the system will affect.*

E.6 Invariants

Comment: *Properties of the environment that the systemâ€™s operation must preserve.*

System

Contents

S.1 Components 17
S.2 Functionality 17
S.3 Interfaces 17
S.4 Detailed usage scenarios 17
S.5 Prioritization 18
S.6 Verification and acceptance criteria 18

Comment: *The System book refines the Goal one by focusing on more detailed requirements about the system under development, mainly its constituents, behaviors and properties.*

S.1 Components

Comment: *Overall structure expressed by the list of major software and, if applicable, hardware parts.*
Comment: *This chapter should not be empty!*

S.2 Functionality

Comment: *One section, S.2.n, for each of the components identified in S.2, describing the corresponding behaviors (functional and non-functional properties).* **Comment:** *This chapter should not be empty!*

S.3 Interfaces

Comment: *How the system makes the functionality of S.2 available to the rest of the world, particularly user interfaces and program interfaces (APIs).*

S.4 Detailed usage scenarios

Comment: *Examples of interaction between the environment (or human users) and the system: use cases, user stories.*

S.5 Prioritization

Comment: *Classification of the behaviors, interfaces and scenarios (S.2, S.3 and S.4) by their degree of criticality.*

S.6 Verification and acceptance criteria

Comment: *Specification of the conditions under which an implementation will be deemed satisfactory.*

Project

Contents

P.1 Roles and personnel 19
P.2 Imposed technical choices 19
P.3 Schedule and milestones 19
P.4 Tasks and deliverables 20
P.5 Required technology elements 22
P.6 Risks and mitigation analysis 22
P.7 Requirements process and report 22

Comment: *The Project book describes all the constraints and expectations not about the system itself, but about how to develop and produce it.*

P.1 Roles and personnel

Comment: *Main responsibilities in the project; required project staff and their needed qualifications.*

P.2 Imposed technical choices

Comment: *Any a priori choices binding the project to specific tools, hardware, languages or other technical parameters.*

P.3 Schedule and milestones

The project's primary schedule will follow the milestones and dates below, each named according to the deliverables they must produce (as discussed in the project's development plan):

- M1 Problem Statement, Development Plan, PoC Plan, and Team Charter (Due by September 28, 2026)
- M2 SRS and Hazard Analysis Revision 0 (Due by October 19, 2026)
- M3 VnV Plan Revision 0 (Due by November 2, 2026)
- M4 Design Document Revision -1 and CD-Fall (FODA) (Due by November 16, 2026)

- M5 PoC Demo (Due by November 23, 2026)
- M6 Design Documentation Revision 0 (Due by January 18, 2027)
- M7 Revision 0 Demo (Due by February 1, 2027)
- M8 VnV Report Revision 0 and CD-Winter Revision 0 (Usability Report) (Due by March 8, 2027)
- M9 Final Demonstration Revision 1, EXPO Demonstration (Due by March 15, 2027)
- M10 Final Documentation (Revision 1) (Due by March 31, 2027)

P.4 Tasks and deliverables

The tasks required to be completed to develop RealmForge (and its feature variation capabilities) will be managed using an iterative and incremental approach. Within each milestone (in this context the milestones associated with source code development), team members will divide the work to be done into suitable issues, viewable in RealmForge's GitHub Project. The issues will be coordinated using a Kanban method, tracking their progress and review status. The software features associated with these issues will be regularly planned, developed, reviewed, tested, and deployed using a feature branch workflow, contributing towards each set milestone.

A schedule of development deliverables pertaining to the milestones listed above is available below, complete with relevant requirement sets that are planned to be implemented at the completion of each deliverable. Milestones that purely involve documentation development are omitted, as this schedule is intended to outline the completion of software development tasks and their associated requirements.

Detailed Schedule:

- M5 PoC Demo
  - **Deadline:** November 23, 2026
  - **Software Deliverables:** RealmForge Proof of Concept Build
    - **Associated Software Tasks:**
    - Review PoC plan and requirements
    - Develop PoC features
    - Review implemented PoC features on a recurring basis during development
    - Test PoC features
    - Verify associated requirements (outlined below)

    - Deploy PoC
  - **High Priority Requirements:** NEED TO ADD
  - **Requirement Sets Implemented:** NEED TO ADD

- M7 Revision 0 Demo
  - **Deadline:** February 1, 2027
  - **Software Deliverables:** RealmForge Revision 0 Build
    - **Associated Software Tasks:**
    - Review Revision 0 design documentation and requirements
    - Develop Revision 0 features
    - Review implemented Revision 0 features on a recurring basis during development
    - Test Revision 0 features
    - Verify associated requirements (outlined below)
    - Deploy Revision 0
  - **High Priority Requirements:** NEED TO ADD
  - **Requirement Sets Implemented:** NEED TO ADD

- M9 Final Demonstration Revision 1, EXPO Demonstration
  - **Deadline:** March 15, 2027
  - **Software Deliverables:** RealmForge Revision 1 Build
    - **Associated Software Tasks:**
    - Review Revision 1 design documentation (in progress) and requirements
    - Develop Revision 1 features
    - Review implemented Revision 1 features on a recurring basis during development
    - Test Revision 1 features
    - Verify associated requirements (outlined below)
    - Deploy Revision 1
  - **High Priority Requirements:** NEED TO ADD
  - **Requirement Sets Implemented:** NEED TO ADD

P.5 Required technology elements

The RealmForge software will be run, developed, and tested using hardware that supports Windows 10/11 OS. Other OS's such as Apple's or other platforms will be considered after the initial development showcase. Inputs are handled either with a standard keyboard and mouse or with a connected controller using standard button layout (analog sticks, d-pads, shoulder and 4-buttons layout). The controller should be compatible with any Windows OS, similarly to the mouse and keyboard. The game output will be displayed using any standard monitor connected to the computer hardware.

The system will be built using the Godot game engine, including using Godot's own coding language, GDScript. Godot is an open-source application and has an easy learning curve to implement features, which is a main concern regarding this project. The easier the team can fully pick up the engine, the faster and cleaner it is to create a working concept. Each team member will choose their own IDE. GDUnit4 and GDLint and other external third-party modules will be used for CI/CD as a safeguard for good coding practices and coverage. The project will use GitHub's version control and cloud storage for the source code and documents. The GDLint and GdUnit4 unit tests and linting will be implemented using GitHub Actions available on the GitHub website; this is so that errors and bugs will be caught and reiterated early on. LaTeX will be used for writing important documents such as Hazard Analysis, Development, etc. Issues about the project will be raised on the GitHub website and written/handled with its own built-in issue tracker. Other important project management tasks will be written using GitHub Project's Kanban board for organization and easy tracking.

P.6 Risks and mitigation analysis

Comment: *Potential obstacles to meeting the schedule of P.4, and measures for adapting the plan if they do arise.*

P.7 Requirements process and report

Comment: *Initially, description of what the requirements process will be; later, report on its steps.*

Appendix

Appendix — Generative AI Use Disclosure

AI Tools Used

- OpenAI GPT-6
- Google Search AI Overview (Google Gemini)

How and Where Used

AI tools were used for assistance with LaTeX syntax and commands, during the development of sections E2, E3, S4, P3, and P4, and when developing LaTeX formatting for the document as a whole. Specifically, AI tools were used for assistance in creating the LaTeX custom requirement format macro used throughout the document, as well as resolving various minor formatting and table of contents bugs. Additionally, these tools were also used in these sections to conduct brief 'sanity checks' of content in accordance with Meyer's requirements document template (the template description was provided). These 'sanity checks' did not substitute full reviews of the sections, and were more used as a brief second opinion. In these sections, AI tools were not used to generate any text or wholly generate any of the ideas or conclusions presented.

Verification of AI-Generated Content

Any LaTeX syntax or code provided by AI tools was verified by testing compilation in LaTeX documents, and ensuring the resulting output was correctly formatted and matched the needs of the document. Conclusions from brief 'sanity checks' of the above sections using AI tools were independently verified for correctness using course materials and documentation of Meyer's requirements template, and no ideas or text were newly generated by AI tools and presented in the document.

Appendix — Reflection

Reflection Questions Answered:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?
4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.
5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.
6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Andreas:

1. When writing this deliverable, adopting Bertrand Meyer's requirement template and completing its sections was helpful and applicable to our project, since all team members were familiar with the template previously, and it is provided with thorough documentation. Understanding the template previously, and being able to review its full documentation helped our team to understand what was required to complete each section. Additionally, writing this deliverable forced our team to deeply consider various details of the project and requirements that the system must satisfy, which helped to enable us to make important project decisions. Obviously one must decide on various project details when writing a requirement specification, but completing this SRS helped us to solidify and exactly outline the outputs needed to be produced by the system, and other constraints it must follow.
2. One pain point I experienced during this deliverable was coordinating the many sections to be written and decisions to be made as a group. Ideally, all project and system planning decisions will be made together, but work must be divided at some point so that each team member can asynchronously complete their tasks (while reviewing and discussing with the team when needed). This at times led to more

administrative overhead work, as various sections needed to be iteratively reviewed and discussed to ensure that the document did not contain major contradictions. However, this coordination work was required for the success of the document, and was lessened in certain scenarios by initially discussing clear expectations for various sections as a group, to ensure that the team member completing a specific section had the information required to do so correctly.

3. A majority of the sections I contributed to did not involve the production of formal requirements, except E.3 (Constraints), which included formal environmental constraint requirements. Some of these requirements (feature variation points, and user input device requirements) were primarily influenced by discussions with our TA and professor, using their recommendations on including feature/system variability in the project, and specifically doing so by having players use different input methods (controller, keyboard and mouse). Additionally, the environment requirement included concerning the operating system RealmForge will support was influenced by discussions with peers that could be included in our user groups (those who play video games and are familiar the roguelite genre, those who are not, etc.). It was found (as expected) that most users tend to play PC video games on machines running modern Windows operating systems, as they are most commonly available and easy to configure to play popular video games.

4. Of the courses I have taken (and are currently taking), SFWRENG 3RA3 (Requirements) and 4HC3 (Human-Computer Interaction) are particularly helpful to our team (and myself specifically) for our capstone project. SFWRENG 3RA3 introduced all team members to the use of Meyer's requirements template and principles of requirements engineering as a whole, which helped us to complete this document, and construct requirements for the system. SFWRENG 4HC3 provides our team with important knowledge and techniques for effective user interface and experience design, which will be particularly helpful for our project, since it primarily focuses on interface design and usability testing.

5. One skill the team will collectively need to acquire (or further develop) is video game development. This will include becoming more familiar with our chosen game development engine and language, as well as better understanding game development principles. This will be especially important for myself, as I have limited experience in programming for video game development. Another area of knowledge that will need to be further developed for each team member concerns usability testing techniques and principles, as each of us are just beginning to study these concepts in SFWRENG 4HC3 (as discussed above).

6. For improving game development skills, team members could use provided game engine tutorials and documentation for the game engine used, review YouTube and web article lessons on game development principles, or read books that provide instruction on game development principles. I will individually aim to primarily follow tutorials and documentation for our chosen game engine, since the initial

technical skills of game development are new to me. I will also aim to improve my understanding of game development principles through various tutorials and lessons when possible. For developing interface design and usability testing knowledge, team members could take notes and review how to best apply the concepts taught in SFWRENG 4HC3 to our Capstone project specifically, or consult other standards or tutorial sources for interface design and usability testing. I will primarily aim to review and understand how the concepts taught in 4HC3 could be applied to our project, as I currently am developing my understanding of these concepts well in the context of the course, and believe they could be effective when applied to the development of RealmForge.

Owen:

1. When writing this deliverable, collaborating using the Meyer template made the process much easier since everyone had worked with it previously in the course SFWRENG 3RA3. The division of work went smoothly at the beginning as well, and communication throughout the process made us get more specific with what exactly implementation would entail, what constraints/requirements exist, and how those will be satisfied.

2. There were some pain points throughout the deliverable. The first one was the adjustment to using branches much more often (as opposed to a branch for each document, it was per section). This took a little bit of getting used to for me to do this without issue, but was resolved by discussing the process with group members, and getting comfortable with the process of branch management. Another pain point was consistent communication, as the bulk of the work was done during reading week, when people's schedules are at their most inconsistent. This was also made worse by many sections being written individually, without realizing at the time of writing that certain sections are similar, which created some redundant points. This was resolved by constantly discussing clear guidelines for sections to ensure everything is cohesive and not repetitive. This preventative measure started once the issue was noticed, and ensured more members knew exactly what was required of them.

3. Basically all requirements I had a hand in writing were directly inspired by speaking to clients/stakeholders, although not always in the context of the project. I have had plenty of discussions about game design with friends and peers that apply to the project and its requirement elicitation, since a lot of friends of mine like games and are interested in game development like I am. Although many of these people are potential users/stakeholders of the project, most of these conversations either predate the project's inception or were not directly related. The exception is our TA and supervisor, both of whom are experienced with video games and stakeholders in the project.

4. I can think of many courses, past and present, that will help the team's success with the capstone project.

- SFWRENG 4HC3: This course is teaching about how to build effective user interfaces and experiences, which will help build usable knowledge for the project since UI and various design principles described in this course are all very important to the success of this project.
- SFWRENG 3RA3: As mentioned in question 1, the group all has experience with the Meyer SRS template introduced in this course and also gained important information about requirements from this course.
- SFWRENG 3S03: Software testing knowledge from this course will be integral to how we approach the project.
- SFWRENG 2XC3: This course was an introduction to many core concepts we use in this project, such as version control, testing, debugging, and CI/CD.

5. The main skill that the team as a whole will need to acquire and develop is video game development and its intricacies. Since I am somewhat knowledgeable about this, I will do what I can to teach the group as needed about the game engine and language we use. Another skill the team will need is setting up test suites and linting. This is at least a requirement for myself, as I am not very experienced with setting these things up, especially in conjunction with a game engine. The practice of good UI design is something being taught in 4HC3 as mentioned in Q4, and is important for the team to absorb from the course.

6. Game Development skill acquisition approaches:
   - Following online tutorials to understand the engine/language and game development techniques, either on YouTube or provided by the engine's documentation.
   - Reading up on documentation to learn what is possible in the chosen engine and language.
   - Discuss with peers or members of the group with more experience with game design and the chosen engine/language.

   Testing and Linting skill acquisition approaches:
   - Experimenting with these tools in a neutral/sample environment.
   - Following online tutorials to set up/use the chosen tools in the chosen game engine.

   Good UI Design skill acquisition approaches:
   - Following along with and drawing from SFWRENG 4HC3 as we are taught more (or consulting a peer in that class, if not taking the class yet) and participating in assignments to build skills.
   - Experimenting with UI setup through YouTube tutorials, 4HC3 knowledge, or other means, tweaking the design based on what you personally like (and testing it on peers).

Humna:

1.
2.
3.
4.
5.
6.

Christopher:

1.
2.
3.
4.
5.
6.

Andy:

1.
2.
3.
4.
5.
6.